

Simulation of simple network using ns 2

Create a simulator object

```
set ns [new Simulator]
```

Open the NAM trace file

```
set nf [open out.nam w]
```

```
$ns namtrace-all $nf
```

Define a finish procedure

```
proc finish {} {
```

```
    global ns nf
```

```
    $ns flush-trace
```

```
    close $nf
```

```
    exec nam out.nam &
```

```
    exit 0
```

```
}
```

Create 2 nodes

```
set n0 [$ns node]
```

```
set n1 [$ns node]
```

Create a duplex link between the nodes

```
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
```

Create a UDP agent and attach it to node n0

```
set udp0 [new Agent/UDP]
```

```
$ns attach-agent $n0 $udp0
```

Create a Null agent (sink) and attach it to node n1

```
set null0 [new Agent/Null]
```

```
$ns attach-agent $n1 $null0
```

```
# Connect the two agents
$ns connect $udp0 $null0

# Create a CBR traffic source and attach it to UDP0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.01
$cbr0 attach-agent $udp0

# Schedule traffic: start at 0.5s and stop at 4.5s
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"

# Schedule the finish procedure
$ns at 5.0 "finish"

# Run the simulation
$ns run
```

Mesh Topology

```
# Create a simulator object
set ns [new Simulator]

# Open the NAM trace file
set nf [open out.nam w]
$ns namtrace-all $nf

# Define a 'finish' procedure
```

```
proc finish {} {  
    global ns nf  
    $ns flush-trace  
    close $nf  
    exec nam out.nam &  
    exit 0  
}
```

```
# Create four nodes
```

```
set n0 [$ns node]
```

```
set n1 [$ns node]
```

```
set n2 [$ns node]
```

```
set n3 [$ns node]
```

```
# Create links between the nodes
```

```
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
```

```
$ns duplex-link $n0 $n2 1Mb 10ms DropTail
```

```
$ns duplex-link $n0 $n3 1Mb 10ms DropTail
```

```
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
```

```
$ns duplex-link $n1 $n3 1Mb 10ms DropTail
```

```
$ns duplex-link $n2 $n3 1Mb 10ms DropTail
```

```
# Create a TCP agent and attach it to node n1
```

```
set tcp0 [new Agent/TCP]
```

```
$tcp0 set class_ 1
```

```
$ns attach-agent $n1 $tcp0
```

```
# Create a TCP Sink agent and attach it to node n3
```

```
set sink0 [new Agent/TCPSink]
```

```
$ns attach-agent $n3 $sink0
```

```
# Connect TCP source and sink
$ns connect $tcp0 $sink0

# Create a CBR traffic source and attach it to TCP
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.01
$cbr0 attach-agent $tcp0

# Schedule the CBR traffic
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"

# Call the finish procedure
$ns at 5.0 "finish"

# Run the simulation
$ns run
```

Bus Topology

```
# Create a simulator object
set ns [new Simulator]

# Open the NAM trace file
set nf [open out.nam w]
$ns namtrace-all $nf

# Define a 'finish' procedure
proc finish {} {
    global ns nf
    $ns flush-trace
```

```
close $nf  
exec nam out.nam &  
exit 0  
}
```

```
# Create five nodes
```

```
set n0 [$ns node]
```

```
set n1 [$ns node]
```

```
set n2 [$ns node]
```

```
set n3 [$ns node]
```

```
set n4 [$ns node]
```

```
# Create a LAN between the nodes
```

```
set lan0 [$ns newLan "$n0 $n1 $n2 $n3 $n4" 0.5Mb 40ms LL Queue/DropTail  
MAC/Csma/CdChannel]
```

```
# Create a TCP agent and attach it to node n1
```

```
set tcp0 [new Agent/TCP]
```

```
$tcp0 set class_ 1
```

```
$ns attach-agent $n1 $tcp0
```

```
# Create a TCP Sink agent and attach it to node n3
```

```
set sink0 [new Agent/TCPSink]
```

```
$ns attach-agent $n3 $sink0
```

```
# Connect the TCP agent to the sink
```

```
$ns connect $tcp0 $sink0
```

```
# Create a CBR traffic source and attach it to TCP
```

```
set cbr0 [new Application/Traffic/CBR]
```

```
$cbr0 set packetSize_ 500
```

```
$cbr0 set interval_ 0.01
$cbr0 attach-agent $tcp0

# Schedule the CBR traffic
$ns at 0.5 "$cbr0 start"
$ns at 4.5 "$cbr0 stop"

# Call the finish procedure at 5 seconds
$ns at 5.0 "finish"

# Run the simulation
$ns run
```

Ring Topology

```
# Create a simulator object
set ns [new Simulator]

# Open the NAM trace file
set nf [open out.nam w]
$ns namtrace-all $nf

# Define a 'finish' procedure
proc finish {} {
    global ns nf
    $ns flush-trace
    close $nf
    exec nam out.nam &
    exit 0
}

# Create six nodes
```

```
set n0 [$ns node]
```

```
set n1 [$ns node]
```

```
set n2 [$ns node]
```

```
set n3 [$ns node]
```

```
set n4 [$ns node]
```

```
set n5 [$ns node]
```

```
# Create duplex links between the nodes
```

```
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
```

```
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
```

```
$ns duplex-link $n2 $n3 1Mb 10ms DropTail
```

```
$ns duplex-link $n3 $n4 1Mb 10ms DropTail
```

```
$ns duplex-link $n4 $n5 1Mb 10ms DropTail
```

```
$ns duplex-link $n5 $n0 1Mb 10ms DropTail
```

```
# Create a TCP agent and attach it to node n1
```

```
set tcp0 [new Agent/TCP]
```

```
$tcp0 set class_ 1
```

```
$ns attach-agent $n1 $tcp0
```

```
# Create a TCP Sink agent and attach it to node n3
```

```
set sink0 [new Agent/TCPSink]
```

```
$ns attach-agent $n3 $sink0
```

```
# Connect the TCP agent with the Sink agent
```

```
$ns connect $tcp0 $sink0
```

```
# Create a CBR traffic source and attach it to the TCP agent
```

```
set cbr0 [new Application/Traffic/CBR]
```

```
$cbr0 set packetSize_ 500
```

```
$cbr0 set interval_ 0.01
```

```
$cbr0 attach-agent $tcp0
```

```
# Schedule the CBR traffic
```

```
$ns at 0.5 "$cbr0 start"
```

```
$ns at 4.5 "$cbr0 stop"
```

```
# Call the finish procedure at 5 seconds
```

```
$ns at 5.0 "finish"
```

```
# Run the simulation
```

```
$ns run
```

Distance vector

```
# Create a simulator object
```

```
set ns [new Simulator]
```

```
# Enable distance-vector routing
```

```
$ns rtproto DV
```

```
# Trace and NAM output files
```

```
set nf [open distance.nam w]
```

```
$ns namtrace-all $nf
```

```
set f0 [open distance.tr w]
```

```
$ns trace-all $f0
```

```
# Finish procedure
```

```
proc finish {} {
```

```
    global ns f0 nf
```

```
    $ns flush-trace
```

```
    close $f0
```

```
    close $nf
```



```
    exec nam distance.nam &  
    exit 0  
}
```

```
# Create 7 nodes
```

```
set n0 [$ns node]
```

```
set n1 [$ns node]
```

```
set n2 [$ns node]
```

```
set n3 [$ns node]
```

```
set n4 [$ns node]
```

```
set n5 [$ns node]
```

```
set n6 [$ns node]
```

```
# Create ring topology links
```

```
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
```

```
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
```

```
$ns duplex-link $n2 $n3 1Mb 10ms DropTail
```

```
$ns duplex-link $n3 $n4 1Mb 10ms DropTail
```

```
$ns duplex-link $n4 $n5 1Mb 10ms DropTail
```

```
$ns duplex-link $n5 $n6 1Mb 10ms DropTail
```

```
$ns duplex-link $n6 $n0 1Mb 10ms DropTail
```

```
# Create a TCP agent (labelled udp0 by you, but using TCP)
```

```
set tcp0 [new Agent/TCP]
```

```
$ns attach-agent $n1 $tcp0
```

```
# Create a TCP sink at node n3
```

```
set sink0 [new Agent/TCPSink]
```

```
$ns attach-agent $n3 $sink0
```

```
# Connect the agents
```

```
$ns connect $tcp0 $sink0
```

```
# Create a CBR application and attach to TCP
```

```
set cbr0 [new Application/Traffic/CBR]
```

```
$cbr0 set packetSize_ 500
```

```
$cbr0 set interval_ 0.005
```

```
$cbr0 attach-agent $tcp0
```

```
# Start/stop CBR traffic
```

```
$ns at 0.05 "$cbr0 start"
```

```
$ns at 4.5 "$cbr0 stop"
```

```
# Simulate link failure and recovery
```

```
$ns rtmodel-at 1.0 down $n1 $n2
```

```
$ns rtmodel-at 2.0 up $n1 $n2
```

```
# Finish the simulation
```

```
$ns at 5.0 "finish"
```

```
# Run the simulation
```

```
$ns run
```

Link state

```
# Create a simulator object
```

```
set ns [new Simulator]
```

```
# Enable Link State routing protocol
```

```
$ns rtproto LS
```

```
# Open trace and NAM files
```

```
set nf [open linkstate.nam w]
```

```
$ns namtrace-all $nf
```

```
set nt [open linkstate.tr w]
```

```
$ns trace-all $nt
```

```
# Define finish procedure
```

```
proc finish {} {
```

```
    global ns nf nt
```

```
    $ns flush-trace
```

```
    close $nf
```

```
    close $nt
```

```
    exec nam linkstate.nam &
```

```
    exit 0
```

```
}
```

```
# Create 7 nodes
```

```
set n0 [$ns node]
```

```
set n1 [$ns node]
```

```
set n2 [$ns node]
```

```
set n3 [$ns node]
```

```
set n4 [$ns node]
```

```
set n5 [$ns node]
```

```
set n6 [$ns node]
```

```
# Create a ring topology
```

```
$ns duplex-link $n0 $n1 1Mb 10ms DropTail
```

```
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
```

```
$ns duplex-link $n2 $n3 1Mb 10ms DropTail
```

```
$ns duplex-link $n3 $n4 1Mb 10ms DropTail
```

```
$ns duplex-link $n4 $n5 1Mb 10ms DropTail
```

```
$ns duplex-link $n5 $n6 1Mb 10ms DropTail
```

```
$ns duplex-link $n6 $n0 1Mb 10ms DropTail
```

Create a UDP agent and attach to n0

set udp0 [new Agent/UDP]

\$ns attach-agent \$n0 \$udp0

Create a Null agent at n3 (traffic sink)

set null0 [new Agent/Null]

\$ns attach-agent \$n3 \$null0

Connect UDP to sink

\$ns connect \$udp0 \$null0

Create CBR traffic source and attach to UDP

set cbr0 [new Application/Traffic/CBR]

\$cbr0 set packetSize_ 500

\$cbr0 set interval_ 0.005

\$cbr0 attach-agent \$udp0

Schedule CBR traffic

\$ns at 0.5 "\$cbr0 start"

\$ns at 4.5 "\$cbr0 stop"

Simulate link failure and recovery

\$ns rtmodel-at 1.0 down \$n1 \$n2

\$ns rtmodel-at 2.0 up \$n1 \$n2

Finish simulation

\$ns at 5.0 "finish"

Run simulation

\$ns run

Hamming Code

```
package com.mycompany.hammingcode;

import java.io.*;
import java.util.*;

class HammingCode {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the 7-bit data code:");

        int d[] = new int[7];

        for (int i = 0; i < 7; i++) {

            d[i] = sc.nextInt();

        }

        int p[] = new int[4];

        // Calculate parity bits

        p[0] = d[0] ^ d[1] ^ d[3] ^ d[4] ^ d[6];
        p[1] = d[0] ^ d[2] ^ d[3] ^ d[5] ^ d[6];
        p[2] = d[1] ^ d[2] ^ d[3];
        p[3] = d[4] ^ d[5] ^ d[6];

        // Create codeword with parity + data bits

        int c[] = new int[11];

        c[0] = p[0];
        c[1] = p[1];
        c[2] = d[0];
        c[3] = p[2];
        c[4] = d[1];
```

```
c[5] = d[2];  
c[6] = d[3];  
c[7] = p[3];  
c[8] = d[4];  
c[9] = d[5];  
c[10] = d[6];
```

```
System.out.println("Complete Code Word is:");  
for (int i = 0; i < 11; i++) {  
    System.out.print(c[i]);  
}  
System.out.println();
```

```
// Receiving code
```

```
System.out.println("Enter the Received code word (11 bits):");  
int r[] = new int[11];  
for (int i = 0; i < 11; i++) {  
    r[i] = sc.nextInt();  
}
```

```
// Extract parity and data bits from received code
```

```
int pr[] = new int[4];  
int rd[] = new int[7];  
pr[0] = r[0];  
pr[1] = r[1];  
rd[0] = r[2];  
pr[2] = r[3];  
rd[1] = r[4];  
rd[2] = r[5];  
rd[3] = r[6];  
pr[3] = r[7];
```

```

rd[4] = r[8];
rd[5] = r[9];
rd[6] = r[10];

// Calculate syndrome bits
int s[] = new int[4];
s[0] = pr[0] ^ rd[0] ^ rd[1] ^ rd[3] ^ rd[4] ^ rd[6];
s[1] = pr[1] ^ rd[0] ^ rd[2] ^ rd[3] ^ rd[5] ^ rd[6];
s[2] = pr[2] ^ rd[1] ^ rd[2] ^ rd[3];
s[3] = pr[3] ^ rd[4] ^ rd[5] ^ rd[6];

// Calculate error position
int dec = (s[0] * 1) + (s[1] * 2) + (s[2] * 4) + (s[3] * 8);

if (dec == 0)
    System.out.println("No error in received code.");
else {
    System.out.println("Error is at position: " + dec);
    // Correct the error
    r[dec - 1] = (r[dec - 1] == 0) ? 1 : 0;
}

System.out.println("Corrected code word is:");
for (int i = 0; i < 11; i++) {
    System.out.print(r[i]);
}

System.out.println();

sc.close();
}
}

```

StopWaitSender

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class StopWaitSender {
    public static void main(String[] args) throws IOException {
        String packet, ack, str, msg = "";
        int n, i = 0, sequence = 0;

        System.out.println("Waiting for connection...");
        ServerSocket ss = new ServerSocket(2004);
        Socket s = ss.accept();
        System.out.println("Connection established.");

        Scanner br = new Scanner(System.in);
        Scanner msgFromReceiver = new Scanner(s.getInputStream());
        PrintStream dos = new PrintStream(s.getOutputStream());

        // Initial message exchange
        str = msgFromReceiver.nextLine();
        System.out.println("Receiver > " + str);

        System.out.println("Enter the data to send: ");
        packet = br.nextLine();
        n = packet.length();

        while (i < n + 1) {
            if (i < n) {
                msg = String.valueOf(sequence);
                msg = msg.concat(packet.substring(i, i + 1));
```



```

    } else if (i == n) {
        msg = "end";
        dos.println(msg);
        break;
    }
    dos.println(msg);

    // Toggle sequence number
    sequence = (sequence == 0) ? 1 : 0;
    System.out.println("Data sent > " + msg);

    System.out.println("Waiting for ACK...");

    ack = msgFromReceiver.nextLine();
    if (ack.equals(String.valueOf(sequence))) {
        i++;
        System.out.println("Receiver > Packet received\n");
    } else {
        // If ACK is lost or incorrect, resend the data
        System.out.println("Timeout! Resending data...");
        sequence = (sequence == 0) ? 1 : 0;
    }
}

System.out.println("All data sent. Exiting.");
s.close();
ss.close();
br.close();
msgFromReceiver.close();
}
}

```

StopWaitReceiver

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class StopWaitReceiver {
    public static void main(String[] args) throws IOException {
        int sequence = 0;
        String data = "";

        // Establish connection with the sender
        Socket s = new Socket("localhost", 2004);
        PrintStream dos = new PrintStream(s.getOutputStream());
        Scanner msgFromSender = new Scanner(s.getInputStream());

        System.out.println("Connection established:");
        dos.println("Connected");

        while (true) {
            String packet = msgFromSender.nextLine();

            if (!packet.equals("end")) {
                if (Integer.parseInt(packet.substring(0, 1)) == sequence) {
                    data += packet.substring(1);
                    sequence = (sequence == 0) ? 1 : 0;
                    System.out.println("\nReceiver > " + packet);
                } else {
                    System.out.println("\nReceiver > " + packet + " (Duplicate Data)");
                }
            }

            // Send acknowledgment
        }
    }
}
```

```

        dos.println(String.valueOf(sequence));

        System.out.println("Data received so far: " + data);
    } else {
        System.out.println("Connection ended.");
        dos.println("Connection ended...");
        s.close();
        break;
    }
}

msgFromSender.close();
dos.close();
}
}

```

Sliding window sender

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class SWSender {

    public static void main(String[] args) throws IOException {

        int SWS = 8; // Sender Window Size

        int LAR = 0; // Last Acknowledgement Received

        int LFS = 0; // Last Frame Sent

        ServerSocket ss = new ServerSocket(500);

        Socket s = ss.accept();

        System.out.println("Connected to Receiver.");

        System.out.println("Type your Message...");
    }
}

```

```
Scanner msgFromReceiver = new Scanner(s.getInputStream());
PrintStream p = new PrintStream(s.getOutputStream());
```

```
while (true) {
    int i = 0, j = 0;

    Scanner msgToReceiver = new Scanner(System.in);
    String t = msgToReceiver.nextLine();

    if (t.trim().toLowerCase().equals("quit")) {
        p.println(t);
        System.exit(0);
    }

    char[] c = t.toCharArray();
    int sent = 1;

    while (i < t.length()) {
        // Sending frames
        while (i < t.length() && (LFS - LAR) < SWS) {
            p.println(c[i]);
            LFS++;
            System.out.println("Sent = " + c[i] + " Successfully");
            i++;
        }

        // Receiving ACKs
        while (j < t.length() && j < SWS * sent) {
            String t1 = msgFromReceiver.nextLine();
            System.out.println("ACK Received: " + t1);
            j++;
        }
    }
}
```

```

        LAR++;
    }

    sent++;
    System.out.println();
}
}
}
}
}

```

Sliding window receiver

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class SWReceiver {
    public static void main(String[] args) throws IOException {
        int RWS = 8; // Receiver Window Size
        int LAF = 0; // Largest Acceptable Frame
        int LFR = 0; // Last Frame Received

        InetAddress obj = InetAddress.getLocalHost();
        Socket s = new Socket(obj, 500);

        Scanner msgFromSender = new Scanner(s.getInputStream());
        PrintStream p = new PrintStream(s.getOutputStream());

        while (true) {
            int i = 0;

            while (i < RWS) {
                if ((LAF - LFR) <= RWS) {

```

```

String t = msgFromSender.nextLine();

if (t.equals("quit")) {
    System.exit(0);
}

System.out.println("Received = " + t + " Successfully");
LFR++;
p.println("Acknowledgement for " + t);
LAF++;
}
i++;
}
System.out.println();
}
}
}

```

TCP1Server

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

class TCP1Server {
    public static void main(String[] args) throws IOException {
        ServerSocket ss = new ServerSocket(8000); // Open the server socket
        System.out.println("Waiting for client connection...");

        Socket s = ss.accept(); // Accept client connection
        System.out.println("Client connected.");

        Scanner in = new Scanner(System.in); // Scanner to read input from keyboard
    }
}

```

```

        PrintStream dos = new PrintStream(s.getOutputStream()); // Send data to client

        while (true) {

            System.out.println("Enter message to send:");

            String str = in.nextLine(); // Read user input
            dos.println(str); // Send message to client

            if (str.equalsIgnoreCase("end")) { // Check for termination

                System.out.println("Connection closed.");
                s.close(); // Close socket
                break;
            }
        }

        ss.close(); // Close server socket after loop ends
    }
}

```

TCP1Client

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

class TCP1Client {

    public static void main(String[] args) throws IOException {

        Socket s = new Socket("localhost", 8000); // Connect to the server

        // Scanner to read data from server input stream
        Scanner in = new Scanner(s.getInputStream());

        while (true) {

            String str = in.nextLine(); // Read message from server

```

```

        System.out.println("Message Received: " + str);

        if (str.equalsIgnoreCase("end")) { // Terminate if "end" is received
            System.out.println("Connection closed by server.");
            s.close(); // Close client socket
            break;
        }
    }
}
}
}

```

Encrypt and Decrypt

```

package sender;

import java.util.*;
import java.io.*;

public class Sender {

    final static int STATE_ARRAY_LENGTH = 256;

    // Key Scheduling Algorithm (KSA)
    public static int[] ksa (String key){

        int[] state = new int[STATE_ARRAY_LENGTH];
        int[] keyTable = new int[STATE_ARRAY_LENGTH];

        // Fill state and key table
        for (int i = 0; i < STATE_ARRAY_LENGTH; i++) {
            state[i] = i;
            keyTable[i] = key.charAt(i % key.length());
        }

        int j = 0;
        for (int i = 0; i < STATE_ARRAY_LENGTH; i++) {
            j = (j + state[i] + keyTable[i]) % STATE_ARRAY_LENGTH;

            // Swap
            int temp = state[i];

```



```

        state[i] = state[j];

        state[j] = temp;
    }

    return state;
}

// Pseudo-Random Generation Algorithm (PRGA)
public static byte[] rc4EncryptDecrypt( byte[] input, int[] state){

    int i = 0, j = 0;

    int[] s = state.clone();

    byte[] output = new byte[input.length];

    for (int k = 0; k < input.length; k++) {

        i = (i + 1) % STATE_ARRAY_LENGTH;

        j = (j + s[i]) % STATE_ARRAY_LENGTH;

        // Swap

        int temp = s[i];
        s[i] = s[j];
        s[j] = temp;

        int t = (s[i] + s[j]) % STATE_ARRAY_LENGTH;

        int keystreamByte = s[t];

        output[k] = (byte) (input[k] ^ keystreamByte);

    }

    return output;
}

} public static void main(String[] args) throws IOException { Scanner scanner = new
Scanner(System.in);

    System.out.print("Enter key (max 5 characters): ");

    String key = scanner.nextLine();

    if (key.length() > 5) {

        key = key.substring(0, 5); // Restrict to 5 chars

    }

    System.out.print("Enter plaintext: ");

    String plainText = scanner.nextLine();

```

```

        byte[] plainBytes = plainText.getBytes();
        int[] stateTable = ksa(key);
    // Encrypt

        byte[] encrypted = rc4EncryptDecrypt(plainBytes, stateTable);

        System.out.print("Encrypted string: ");

        for (byte b : encrypted) {
            System.out.print((char) b + " ");
        }

        System.out.println();

    // Decrypt

        byte[] decrypted = rc4EncryptDecrypt(encrypted, stateTable);

        System.out.print("Decrypted string: ");

        for (byte b : decrypted) {
            System.out.print((char) b);
        }

        System.out.println();

        scanner.close();

    }

}

```

Echo Server

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class EchoServer {

    public static void main(String[] args) throws IOException {

        ServerSocket ss = new ServerSocket(500); // Create server socket on port 500

        System.out.println("Server is ready and waiting for client...");
    }
}

```

```

Socket s = ss.accept(); // Accept client connection

System.out.println("Client connected.");

Scanner msgFromClient = new Scanner(s.getInputStream()); // Read from client
PrintStream p = new PrintStream(s.getOutputStream()); // Send to client

while (true) {
    if (!msgFromClient.hasNextLine()) {
        break; // Exit if client disconnects
    }

    String t = msgFromClient.nextLine(); // Read client's message

    System.out.println("Received: " + t); // Print it to server console
    p.println(t); // Echo back to client

    if (t.equalsIgnoreCase("exit")) {
        System.out.println("Connection closed.");
        break;
    }
}

// Close all resources
msgFromClient.close();
p.close();
s.close();
ss.close();
}
}
Echo Client
import java.io.*;

```

```

import java.net.*;
import java.util.Scanner;

public class EchoClient {
    public static void main(String[] args) throws IOException {
        InetAddress obj = InetAddress.getLocalHost();
        Socket s = new Socket(obj, 500); // Connect to server on port 500

        Scanner msgFromServer = new Scanner(s.getInputStream()); // Read from server
        PrintStream p = new PrintStream(s.getOutputStream()); // Write to server

        System.out.println("TYPE YOUR MESSAGE TO SERVER AND TYPE 'quit' TO EXIT");

        Scanner userInput = new Scanner(System.in); // Read from console

        while (true) {
            String t = userInput.nextLine(); // Get user input

            if (t.equalsIgnoreCase("quit")) {
                p.close();
                s.close();
                System.exit(0);
            } else {
                p.println(t); // Send to server
                String response = msgFromServer.nextLine(); // Receive echo
                System.out.println("From Server: " + response);
            }
        }
    }
}

```

PingServer

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

class PingServer {
    public static void main(String args[]) {
        try {
            String str;

            // Get IP from user
            System.out.print("Enter the IP Address to be pinged: ");
            Scanner s1 = new Scanner(System.in);
            String ip = s1.nextLine();

            // Execute ping command
            Runtime runtime = Runtime.getRuntime();
            Process process = runtime.exec("ping " + ip);

            // Read and print ping output
            Scanner s2 = new Scanner(process.getInputStream());
            while (s2.hasNextLine()) {
                str = s2.nextLine();
                System.out.println(" " + str);
            }

            s2.close();
            s1.close();
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

```

}

TCP2server

import java.io.*;

import java.net.*;

import java.util.Scanner;

class Tcp2Server {

    public static void main(String[] args) throws IOException {

        // Create a server socket listening on port 8000

        ServerSocket ss = new ServerSocket(8000);

        System.out.println("Server is waiting for client connection...");

        // Accept the client connection

        Socket s = ss.accept();

        System.out.println("Client connected.");

        // Setup streams for communication

        PrintStream dos = new PrintStream(s.getOutputStream());

        Scanner msgToSend = new Scanner(System.in); // input from server's keyboard

        Scanner msgFromClient = new Scanner(s.getInputStream()); // input from client

        while (true) {

            System.out.print("Enter the message to send: ");

            String str = msgToSend.nextLine(); // server types message

            dos.println(str); // send to client

            if (str.equals("end")) {

                System.out.println("Connection closed by server.");

                s.close();

                ss.close();

                break;
            }
        }
    }
}

```

```

    }

    // Read reply from client
    String str1 = msgFromClient.nextLine();
    System.out.println("Message received: " + str1);

    if (str1.equals("end")) {
        System.out.println("Connection closed by client.");
        s.close();
        ss.close();
        break;
    }
}

msgToSend.close();
msgFromClient.close();
}
}

TCP2Client
import java.io.*;
import java.net.*;
import java.util.Scanner;

class Tcp2Client {
    public static void main(String[] args) throws IOException {
        // Connect to server at localhost, port 8000
        Socket s = new Socket("localhost", 8000);
        System.out.println("Connected to server.");

        // Create input/output streams
        Scanner msgFromServer = new Scanner(s.getInputStream()); // Read from server

```

```

Scanner msgToSend = new Scanner(System.in);          // Read from user input
PrintStream dos = new PrintStream(s.getOutputStream()); // Send to server

while (true) {
    // Read message from server
    String str = msgFromServer.nextLine();
    System.out.println("Message received: " + str);

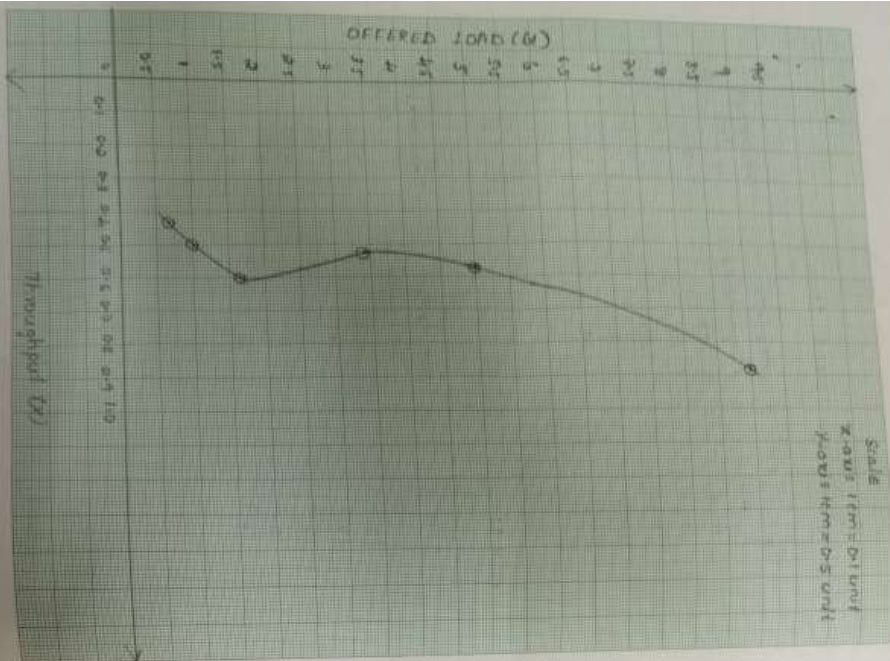
    if (str.equals("end")) {
        System.out.println("Server closed the connection.");
        s.close();
        break;
    }

    // Prompt user to enter reply
    System.out.print("Enter the message to send: ");
    String str1 = msgToSend.nextLine();
    dos.println(str1); // Send to server

    if (str1.equals("end")) {
        System.out.println("Connection closed by client.");
        s.close();
        break;
    }
}

msgFromServer.close();
msgToSend.close();
}
}

```

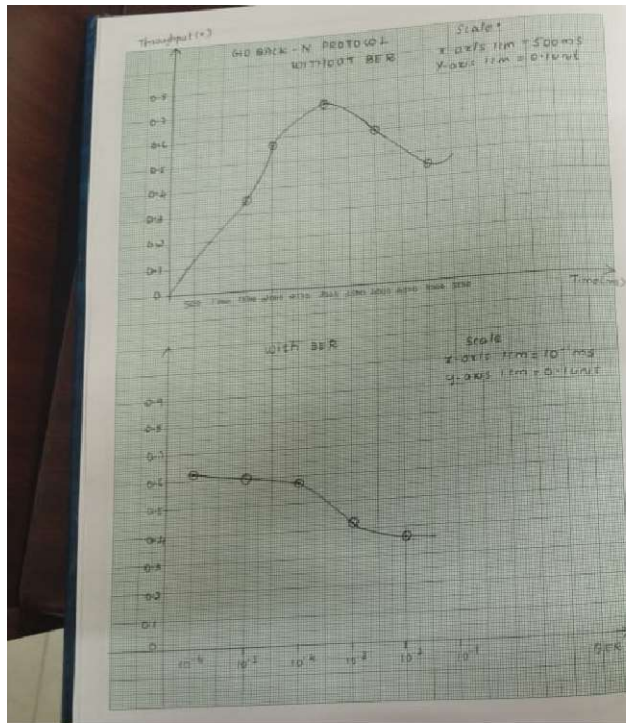


Expt: 5
Plator

Packet Length = 1000 Duration = 30 s Packet Delay = 200 ms

Inter Packet Delay (ms)	Successfully Transmitted				Average Delay				Offered Load (G)				Avg Offered Load	Throughput (G)
100	15	33	18	39	1120	911	1529	768	708	367	274	274	9.24	0.835
200	17	15	15	22	1870	2001	1839	1873	1673	105	124	120	5.1	0.575
300	14	20	20	12	2872	1122	1439	2476	51	74	152	141	3.486	0.525
800	22	11	12	23	1387	3032	1661	1307	34	40	44	30	1.63	0.6
1000	16	12	13	17	1373	1863	2307	1663	30	29	32	35	1.05	0.546
2000	13	11	16	11	2154	2308	2020	2528	27	17	50		0.553	0.441

Implementation and study of token bucket



STUDY AND IMPLEMENTATION OF GO BACK - N AND SELECTIVE REPEAT PROTOCOLS

Observation:

Packet Length = 1000 byte

Data rate = 8 kbps

Duration = 80 sec

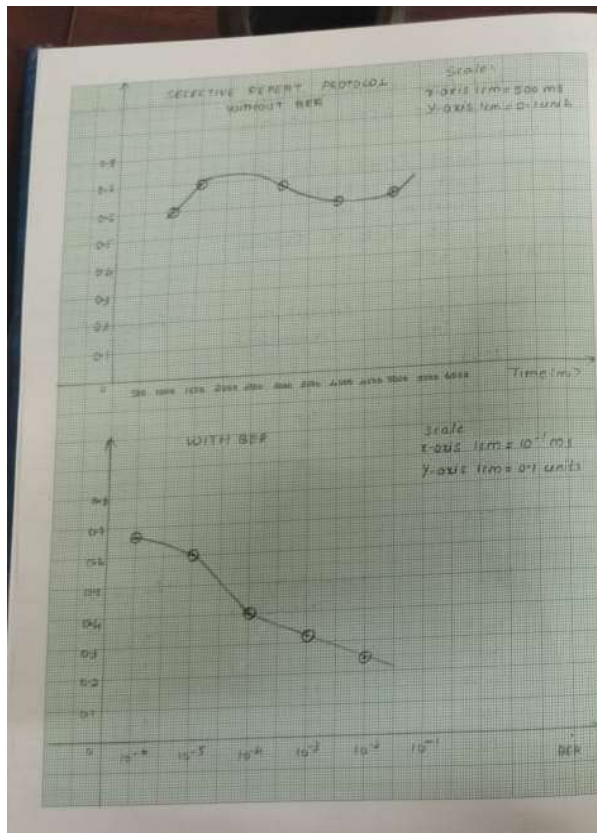
GO BACK - N PROTOCOL

WITHOUT BER

Time-out (ms)	Successfully Transmitted	Throughput (x)
1500	11	0.38
2000	17	0.58
3000	42	0.73
4000	18	0.6
5000	18	0.38

WITH BER

BER	Successfully Transmitted	Throughput (x)
10^{-6}	19	0.63
10^{-5}	18	0.6
10^{-4}	17	0.52
10^{-3}	13	0.43
10^{-2}	11	0.37

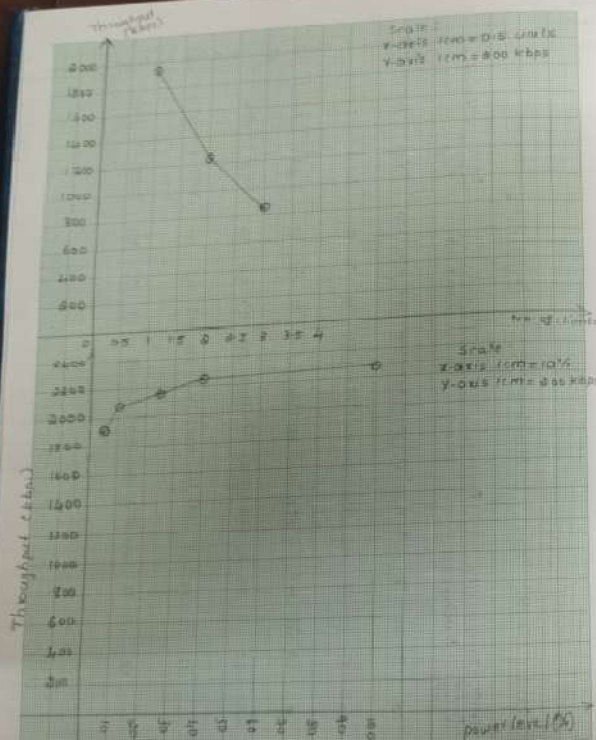


SELECTIVE REPEAT PROTOCOL

Time-out (ms)	Successfully Transmitted	Throughput (x)
1500	19	0.63
2000	20	0.68
3000	20	0.68
4000	18	0.6
4500	17	0.62

WITH BER:

BER	Successfully Transmitted	Throughput (x)
10^{-6}	20	0.63
10^{-5}	18	0.6
10^{-4}	13	0.412
10^{-3}	10	0.33
10^{-2}	8	0.25



Exp No. 6
 22-12-20

Performance Analysis of Wireless LAN (Transmission)

Observation

No. of clients vs. Throughput

No. of clients	Throughput (Kbps)
1	1996.32
2	1709.30
3	1681.76

Power level vs. Throughput

Power level	Throughput (Kbps)
5%	1709.30
10%	2081.30
25%	2181.76
50%	2211.67
100%	2275.70